

LAB ASSIGNMENT – 9.5

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Problem 1: String Utilities Function

Consider the following Python function:

```
def reverse_string(text):  
    return text[::-1]
```

Task:

1. Write documentation in:

- o (a) Docstring
- o (b) Inline comments
- o (c) Google-style documentation

2. Compare the three documentation styles.

3. Recommend the most suitable style for a utility-based string library.

Prompt:

#Write Python documentation for a string utility function using docstring, inline comments, and Google-style format, compare them, and recommend the best style.

```
1 #Write Python documentation for a string utility function using docstring, inline comments, and Google-style format, compare them, and recommend the best style.
2 def reverse_string(text):
3     | return text[::-1]
4 # Docstring style:
5 def reverse_string(text):
6     """
7     Reverses the given string.
8
9     Args:
10     | text (str): The string to be reversed.
11
12     Returns:
13     | str: The reversed string.
14     """
15     return text[::-1]
16 # Inline comments style:
17 def reverse_string(text):
18     | # Reverse the input string using slicing
19     | return text[::-1]
20 # Google-style format:
21 def reverse_string(text):
22     """Reverses the given string.
23
24     Args:
25     | text (str): The string to be reversed.
26
27     Returns:
28     | str: The reversed string.
29     """
30     return text[::-1]
31 # Comparison:
32 # The docstring style provides a clear and structured way to document the function, including the purpose, arguments, and return value. It is widely used in Python and is supported by m
33 # The inline comments style is less formal and may not provide as much information about the function's purpose and usage. It is more suitable for explaining specific lines of code rather
```

```
PS C:\Users\boora\Downloads\3-2\AI-AC> "1111"
1111
PS C:\Users\boora\Downloads\3-2\AI-AC> "avara"
avara
PS C:\Users\boora\Downloads\3-2\AI-AC> "bye"
bye
PS C:\Users\boora\Downloads\3-2\AI-AC> []
```

Problem 2: Password Strength Checker

Consider the function:

```
def check_strength(password):
```

```
    return len(password) >= 8
```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

Prompt:

#Write documentation for a Python password strength checker function using basic docstring, inline comments, and Google-style format, then compare the styles and recommend the most suitable one for security-related code.

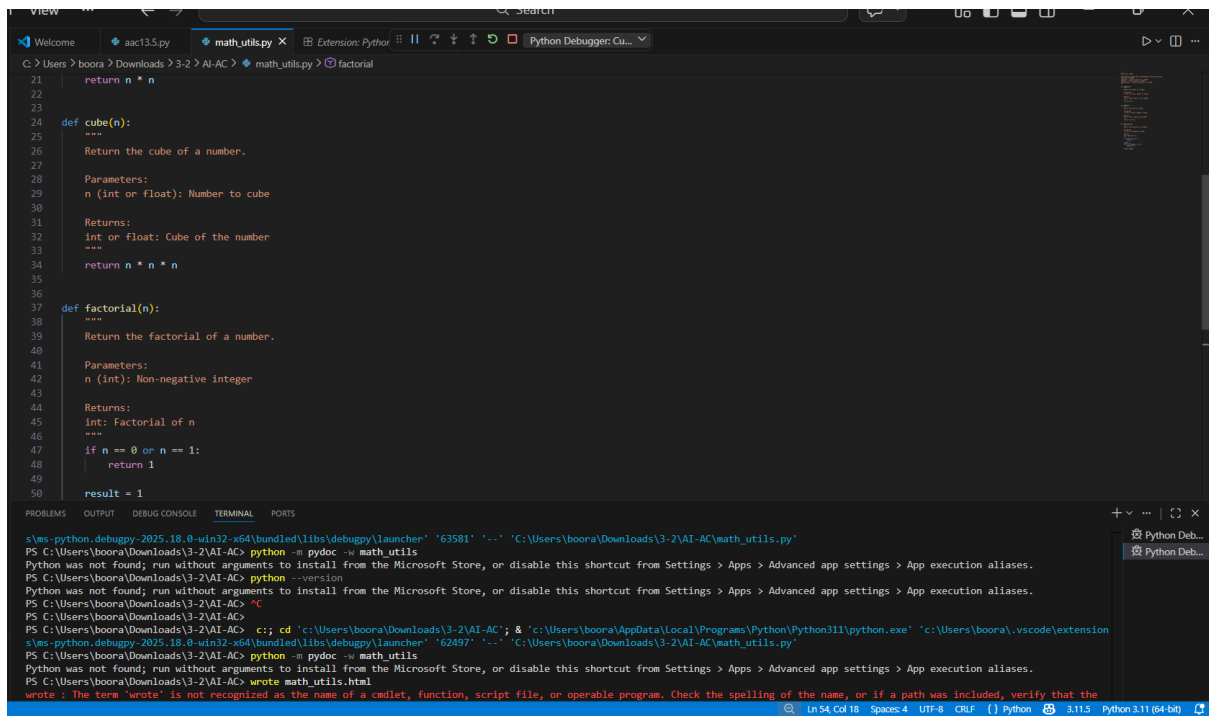
```
1 #Write documentation for a Python password strength checker function using basic docstring, inline comments, and Google-style format, then compare the styles and recommend the most suitable.
2 def check_strength(password):
3     return len(password) >= 8
4 # Basic docstring style
5 def check_strength_basic(password):
6     """Check the strength of a password.
7
8     A password is considered strong if it has at least 8 characters.
9
10    Args:
11        password (str): The password to check.
12    Returns:
13        bool: True if the password is strong, False otherwise.
14    """
15    return len(password) >= 8
16 # inline comments style
17 def check_strength_inline(password):
18     # Check if the password has at least 8 characters
19     return len(password) >= 8
20 # Google-style docstring
21 def check_strength_google(password):
22     """Check the strength of a password.
23
24     A password is considered strong if it has at least 8 characters.
25
26    Args:
27        password (str): The password to check.
28    Returns:
29        bool: True if the password is strong, False otherwise.
30    """
31    return len(password) >= 8
32 # Comparison of styles
33 # The basic docstring style provides a clear and concise description of the function, its arguments, and return value. It is suitable for most cases and is widely used in the Python community.
```

```
PS C:\Users\boora\Downloads\3-2\AI-AC> "bye"
bye
PS C:\Users\boora\Downloads\3-2\AI-AC> ^C
PS C:\Users\boora\Downloads\3-2\AI-AC>
PS C:\Users\boora\Downloads\3-2\AI-AC> cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '51951' '-...' 'C:\Users\boora\Downloads\3-2\AI-AC\aac9.5.py'
True
False
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Problem 3: Math Utilities Module

Task:

1. Create a module `math_utils.py` with functions:
 - o `square(n)`
 - o `cube(n)`
 - o `factorial(n)`
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.



```
21     return n * n
22
23
24 def cube(n):
25     """
26     Return the cube of a number.
27
28     Parameters:
29     n (int or float): Number to cube
30
31     Returns:
32     int or float: Cube of the number
33     """
34     return n * n * n
35
36
37 def factorial(n):
38     """
39     Return the factorial of a number.
40
41     Parameters:
42     n (int): Non-negative Integer
43
44     Returns:
45     int: Factorial of n
46     """
47     if n == 0 or n == 1:
48         return 1
49     result = 1
50     for i in range(1, n + 1):
51         result *= i
52     return result
```

```
s:\ms-python.debuggy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher '63581' '-' 'C:\Users\boora\Downloads\3-2\AI-AC\math_utils.py'
PS C:\Users\boora\Downloads\3-2\AI-AC> python -m pydoc -w math_utils
Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases.
PS C:\Users\boora\Downloads\3-2\AI-AC> python --version
Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases.
PS C:\Users\boora\Downloads\3-2\AI-AC> cd
PS C:\Users\boora\Downloads\3-2\AI-AC> cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extension
s\ms-python.debuggy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '62497' '-' 'C:\Users\boora\Downloads\3-2\AI-AC\math_utils.py'
PS C:\Users\boora\Downloads\3-2\AI-AC> python -m pydoc -w math_utils
Python was not found; run without arguments to install from the Microsoft Store, or disable this shortcut from Settings > Apps > Advanced app settings > App execution aliases.
PS C:\Users\boora\Downloads\3-2\AI-AC> write math_utils.html
write : The term 'write' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the
```

Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:

- o mark_present(student)
- o mark_absent(student)
- o get_attendance(student)

2. Add proper docstrings.

3. Generate and view documentation in terminal and browse

```

Welcome  math_utils.py  attendance.py X  Python Debugger: Cu...
C:\Users\boora> Downloads\3-2\AI-AC> attendance.py get_attendance
1  """
2  attendance module
3
4  This module manages student attendance.
5  """
6
7  attendance = {}
8
9
10 def mark_present(student):
11     """Mark a student as present."""
12     attendance[student] = "Present"
13
14
15 def mark_absent(student):
16     """Mark a student as absent."""
17     attendance[student] = "Absent"
18
19
20 def get_attendance(student):
21     """Return attendance status of a student."""
22     return attendance.get(student, "No record")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\boora\Downloads\3-2\AI-AC>
PS C:\Users\boora\Downloads\3-2\AI-AC> c:; cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extension' Python Deb...